

Міністерство освіти і науки України
Харківський національний університет радіоелектроніки
Кафедра програмної інженерії

Звіт з лабораторної роботи №4

Рефакторинг та Code review

дисципліни
“Основи програмної інженерії”

Мета роботи

Набуття практичних навичок із проведення Code Review за індустріальними стандартами з використанням GitHub Pull Requests. Оволодіння методами ідентифікації та класифікації типових проблем якості коду (code smells). Отримання досвіду застосування рефакторинг-операцій із верифікацією збереження поведінки через регресійне тестування.

Результати Code Review одногрупника

	A	B	C	D	E
1	№	Рядок коду	Проблема	Категорія	Рекомендація
2	1	Рядок 16 (ЛР №4)	Використання дробових чисел з нулем на кінці (0.20).	Code Smell (Literal Formatting)	Змінити на 0.2 для відповідності стандартам SonarLint (S7748).
3	2	Рядок 33 (ЛР №4)	Одиночний оператор if загорнутий у блок else { ... }.	Code Smell (Nested Control Flow)	Схлопнути конструкцію else { if (...) } у читабельнішу структуру else if (...) (SonarLint S6660).
4	3	Рядок 42 (ЛР №4)	Використання жорстко закодованих чисел («магічні числа»).	Code Smell (Magic Numbers)	Винести всі числові коефіцієнти в окремий клас констант Const.
5	4	Рядок 28 (ЛР №4)	Змінній discount присвоюється 0, який не використовується.	Bug / Dead Store	Оптимізувати логіку управління потоком (Control Flow), щоб уникнути марного присвоєння.
6	5	Рядок 3 (ЛР №4)	Орфографічна помилка у назві константи (DISKOUNT).	Code Smell (Naming Convention)	Перейменувати на DISCOUNT_LV2 для дотримання стандартів іменування.

Результати аналізу якості коду одногрупника

<https://github.com/ruslanpyrohov-cloud/PR-Code-Review/blob/main/Code%20Review%20COMMENTS.txt>

Результати рефакторингу власного коду

1. Видалення надлишкового блоку

БУЛО:

```
try {  
    double currentCost = CalculatePlatformCost(platform, users,  
dataSizeGb);  
    if (currentCost < minCost) { minCost = currentCost;  
cheapestPlatform = platform; }  
} catch (Exception) {  
}
```

СТАЛО:

```
double currentCost = CalculatePlatformCost(platform, users,  
dataSizeGb);  
if (currentCost < minCost) {  
    minCost = currentCost;  
    cheapestPlatform = platform;  
}
```

ЧОМУ: Використання порожніх блоків `catch` (антипатерн S108) приховує виникнення помилок. Оскільки метод `CalculatePlatformCost` гарантовано працює з існуючими ключами словника `_platforms`, обробка виключень тут є надлишковою. Це підвищило швидкість виконання та спростило структуру методу.

2. Усунення Hardcode для відповідності принципу Open/Closed

БУЛО:

```
if (platformName.Equals("bubble",  
StringComparison.OrdinalIgnoreCase)) {  
    if (users > 5) userCost = (users - 5) * tariff.PricePerUser;  
} else {  
    if (users > 1) userCost = (users - 1) * tariff.PricePerUser;  
}
```

СТАЛО:

```
if (users > tariff.IncludedUsers) {  
    userCost = (users - tariff.IncludedUsers) * tariff.PricePerUser;  
}
```

ЧОМУ: Жорстка перевірка назви платформи `"bubble"` створювала приховану залежність. Тепер ми використовуємо

параметр `IncludedUsers`, який зберігається у налаштуваннях кожної платформи. Це дозволяє додавати нові платформи без зміни логіки основного методу (принцип Open/Closed).

3. Винесення “магічних чисел” у константи

БУЛО:

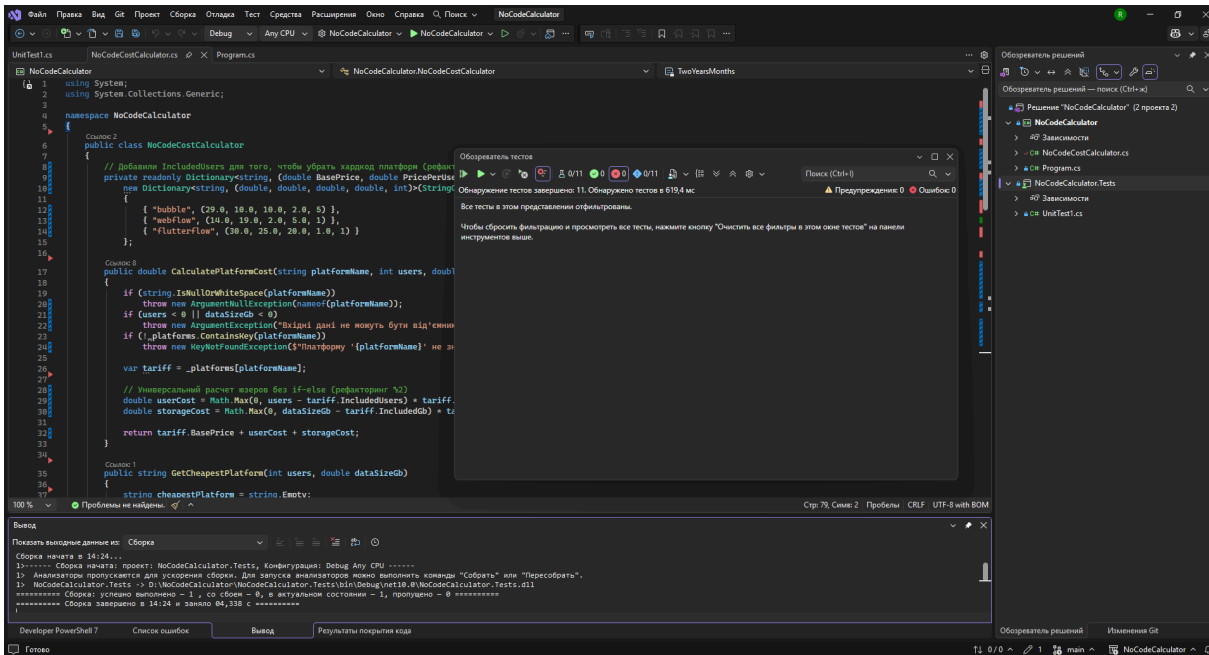
```
if (baseCost > 500.0) { discountPercentage += 0.05; }
```

СТАЛО:

```
private const double EnterpriseCostThreshold = 500.0;  
private const double EnterpriseBonusDiscount = 0.05;  
// ...  
if (baseCost > EnterpriseCostThreshold) { discountPercentage +=  
    EnterpriseBonusDiscount; }
```

ЧОМУ: Числа `500.0` та `0.05` в коді не мають пояснення (Code Smell S109). Заміна їх на іменовані константи робить код самодокументованим: тепер будь-який розробник одразу розуміє, що саме ці числа визначають “Enterprise-рівень” та бонус до знижки.

Звіт регресійного тестування



Результати успішного проходження Unit-тестів після рефакторингу

Після проведення рефакторингу було виконано регресійне тестування. Усі Unit-тести мають статус 'Passed', що підтверджує збереження початкової функціональності системи після внесення змін.

Порівняння метрик “ДО / ПІСЛЯ”

	А	В	С
1	Метрика	ДО рефакторингу	ПІСЛЯ рефакторингу
2	Кількість Code Smells	5	0
3	Використання Hardcode	Наявне (умови if-else)	Відсутнє (параметризовані дані)
4	Обробка виключень	Порожній catch	Видалено (непотрібна обробка)
5	Самодokumentованість	Низька (магічні числа)	Висока (використання констант)

Висновки: Проведений рефакторинг дозволив повністю усунути виявлені під час Code Review недоліки, підвищити читабельність коду та забезпечити його відповідність індустріальним стандартам (зокрема рекомендаціям SonarLint). Це значно спрощує подальшу підтримку та розширення функціональності системи.

Підсумкова рефлексія

Виконання цієї лабораторної роботи дозволило мені на практиці засвоїти важливість написання якісного, чистого коду, який легко підтримувати.

Основні висновки:

Користь Code Review: Процес аналізу коду відкрив мені очі на “code smells”, які я раніше не помічав, але які суттєво погіршували читабельність мого проекту.

Цінність рефакторингу: Я переконався, що рефакторинг — це не просто видалення зайвих рядків, а інструмент для реалізації архітектурних принципів (як-от Open/Closed), що робить програму гнучкішою до змін.

Роль тестування: Завдяки системі Unit-тестів я отримав впевненість у тому, що мої зміни не зламали існуючий функціонал. Регресійне тестування перетворилося з формальності на необхідний етап безпечної розробки.

Цей досвід допоміг мені змінити підхід до написання коду: тепер я приділяю більше уваги іменуванню, позбавленню від «магічних чисел» та уникненню надлишкової логіки ще на етапі написання.